LAPORAN PRAKTIKUM REKAYASA ARSITEKTUR & BASIS DATA

IMPLEMENTASI FUNCTION DAN TRIGGER

Disusun Untuk Memenuhi Salah Satu Tugas Mata Kuliah Prak Rekaysa & Arsitektur Basis Data
Pertemuan 5

Dosen pengampu :

Cecep kurnia satradipraja, S.Kom., M.kom.



Disusun Oleh:

Niha Nuraeni 2507042

JURUSAN ILMU KOMPUTER

PROGRAM STUDI SISTEM INFORMASI

INSTITUTE TEKNOLOGI GARUT

2026

Tugas mandiri menggunakan pola function, trigger, parameter, percabangan, dan pencatatan log yang telah dibahas di kelas. Mahasiswa mengembangkan solusi dengan skenario baru**.**

Tugas 1. Function Perhitungan Pajak

Bekal: Percobaan function hitung_total dan hitung_diskon.

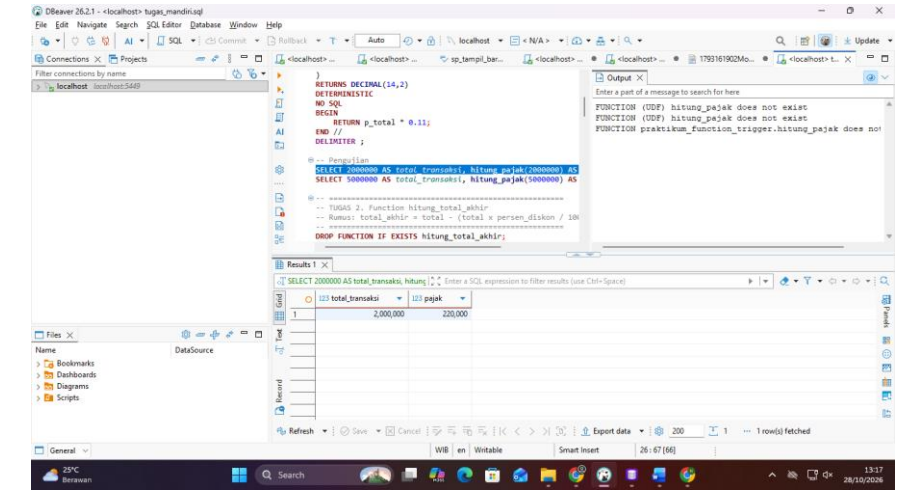
Buat function hitung_pajak yang menerima total transaksi dan mengembalikan pajak sebesar 11%.

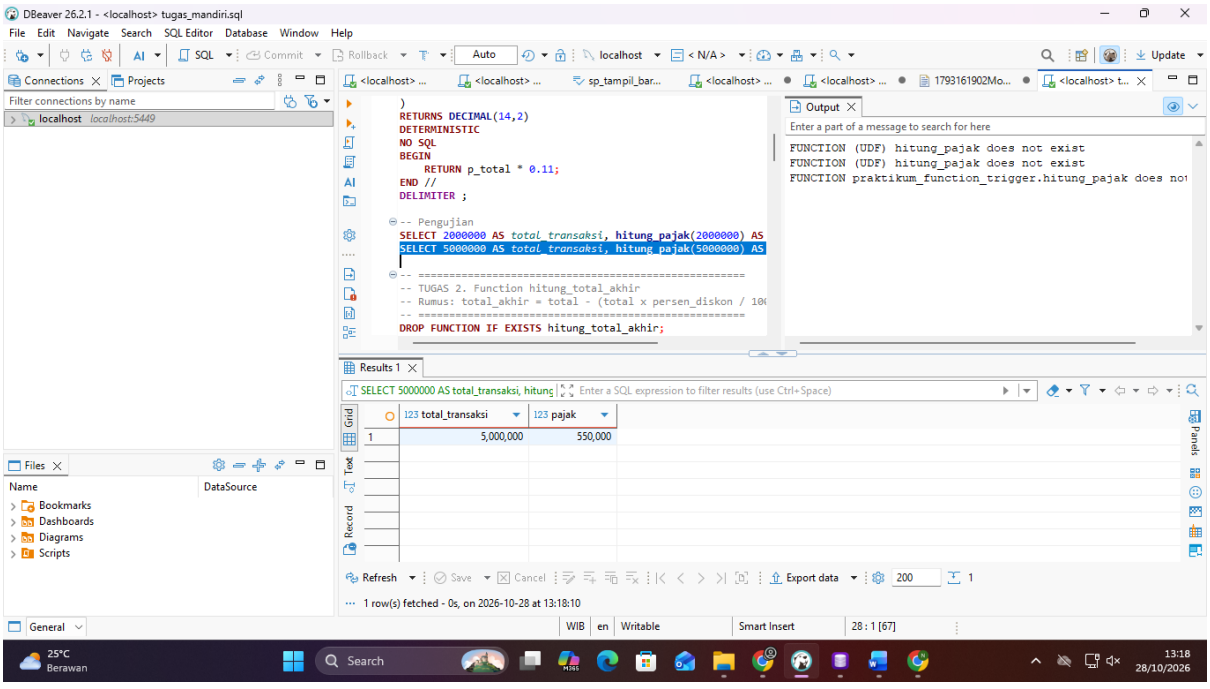
Uji menggunakan data berikut.

```
DROP FUNCTION IF EXISTS hitung_pajak;

DELIMITER //

CREATE FUNCTION hitung_pajak(
    p_total DECIMAL(14,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_total * 0.11;
END //
DELIMITER ;
```





Tugas 2. Function Total Akhir

Bekal: Percobaan function diskon dan penggabungan function.

Buat function hitung_total_akhir yang menerima total transaksi dan persentase diskon.

Rumus:

$$\text{Total Akhir} = \text{Total Transaksi} \times \left(1 - \frac{\text{Diskon}}{100}\right)$$

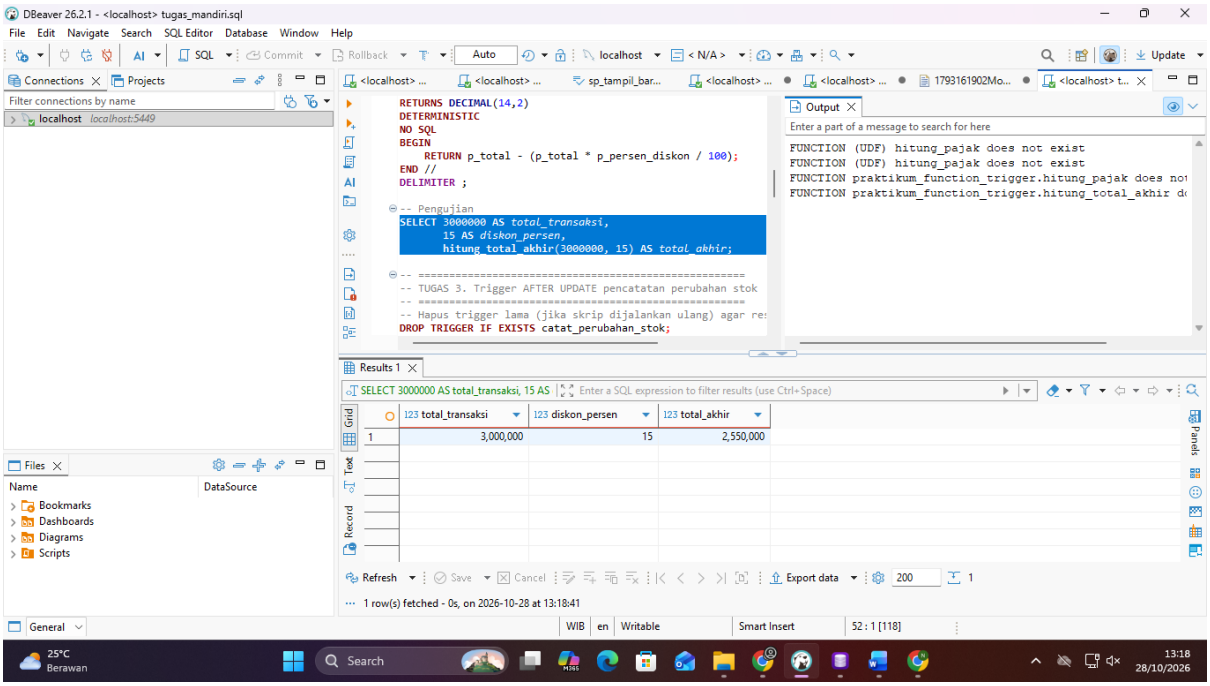
Uji dengan total transaksi Rp3.000.000 dan diskon 15%.

```
DROP FUNCTION IF EXISTS hitung_total_akhir;


DELIMITER //
CREATE FUNCTION hitung_total_akhir(
   p_total DECIMAL(14,2),
   p_persen_diskon DECIMAL(5,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
   RETURN p_total - (p_total * p_persen_diskon / 100);
END //
DELIMITER ;
```

PENGUJIAN

```
SELECT 3000000 AS total_transaksi,
      15 AS diskon_persen,
      hitung_total_akhir(3000000, 15) AS total_akhir;
```



Tugas 3. Trigger Pencatatan Perubahan Stok

Bekal: Trigger AFTER INSERT, tabel log_stok, serta konsep OLD dan NEW.

Buat trigger AFTER UPDATE pada tabel produk untuk mencatat perubahan stok.

Ketentuan:

1. Catat stok sebelum perubahan menggunakan OLD.stok.
2. Catat stok setelah perubahan menggunakan NEW.stok.
3. Simpan ID produk, stok awal, stok akhir, keterangan, dan waktu perubahan.
4. Uji perubahan stok Mouse dari 25 menjadi 30 unit.

Jika stok telah berubah akibat praktikum sebelumnya, atur kembali stok ke 25 sebelum pengujian dan pastikan tidak ada transaksi pengujian yang mengganggu hasil.

```
DELIMITER //

CREATE TRIGGER catat_perubahan_stok

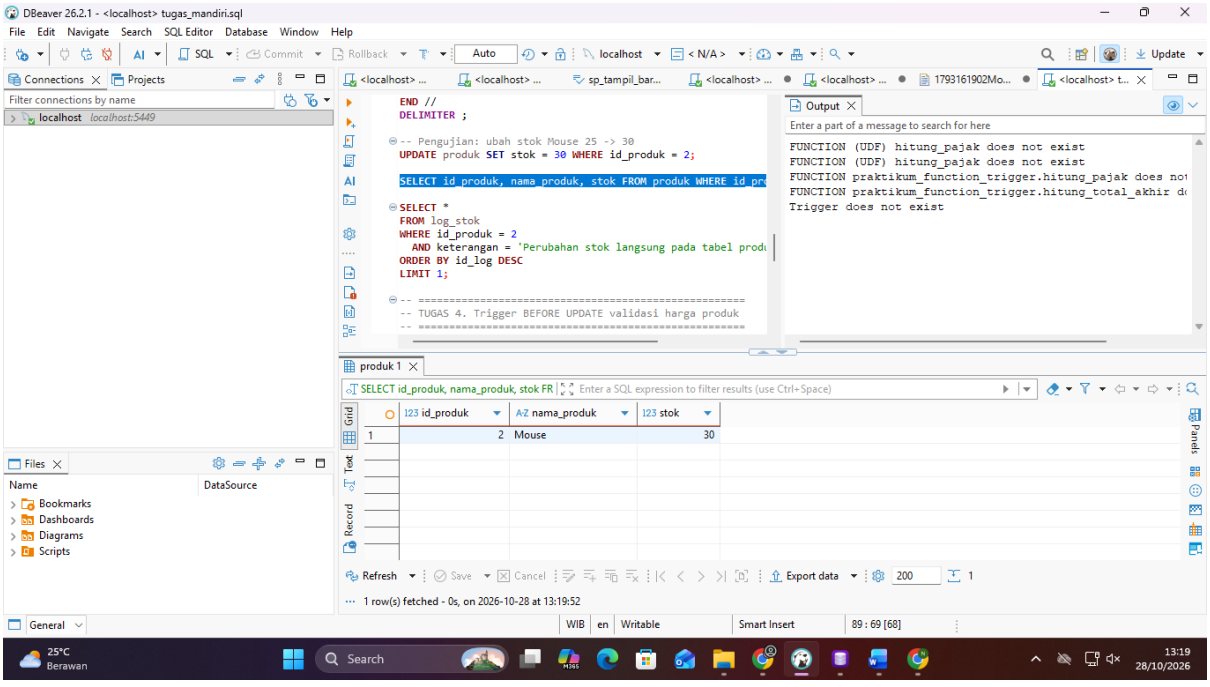
AFTER UPDATE ON produk

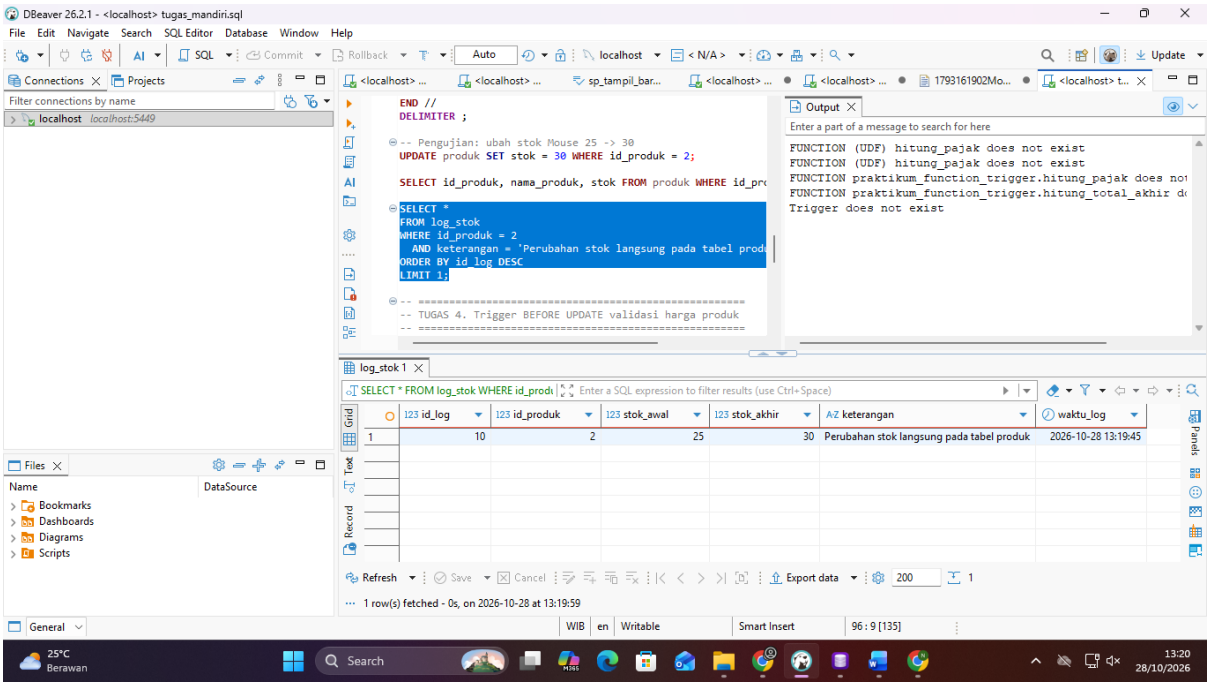
FOR EACH ROW

BEGIN

    IF OLD.stok <> NEW.stok THEN
```

```
        INSERT INTO log_stok (
            id_produk,
            stok_awal,
            stok_akhir,
            keterangan,
            waktu_log
        )
        VALUES (
            NEW.id_produk,
            OLD.stok,
            NEW.stok,
            'Perubahan stok langsung pada tabel produk',
            CURRENT_TIMESTAMP
        );
    END IF;
END // DELIMITER ;
```





Tugas 4. Trigger Validasi Harga Produk

Bekal: Trigger BEFORE INSERT dan penggunaan SIGNAL SQLSTATE '45000'.

Buat trigger BEFORE UPDATE pada tabel produk untuk menolak perubahan harga menjadi nol atau negatif.

Uji dua skenario:

- Mengubah harga Mouse menjadi Rp200.000: perubahan diterima.
- Mengubah harga Mouse menjadi Rp0: perubahan ditolak.

```
DROP TRIGGER IF EXISTS validasi_harga_produk;


DELIMITER //

CREATE TRIGGER validasi_harga_produk

BEFORE UPDATE ON produk

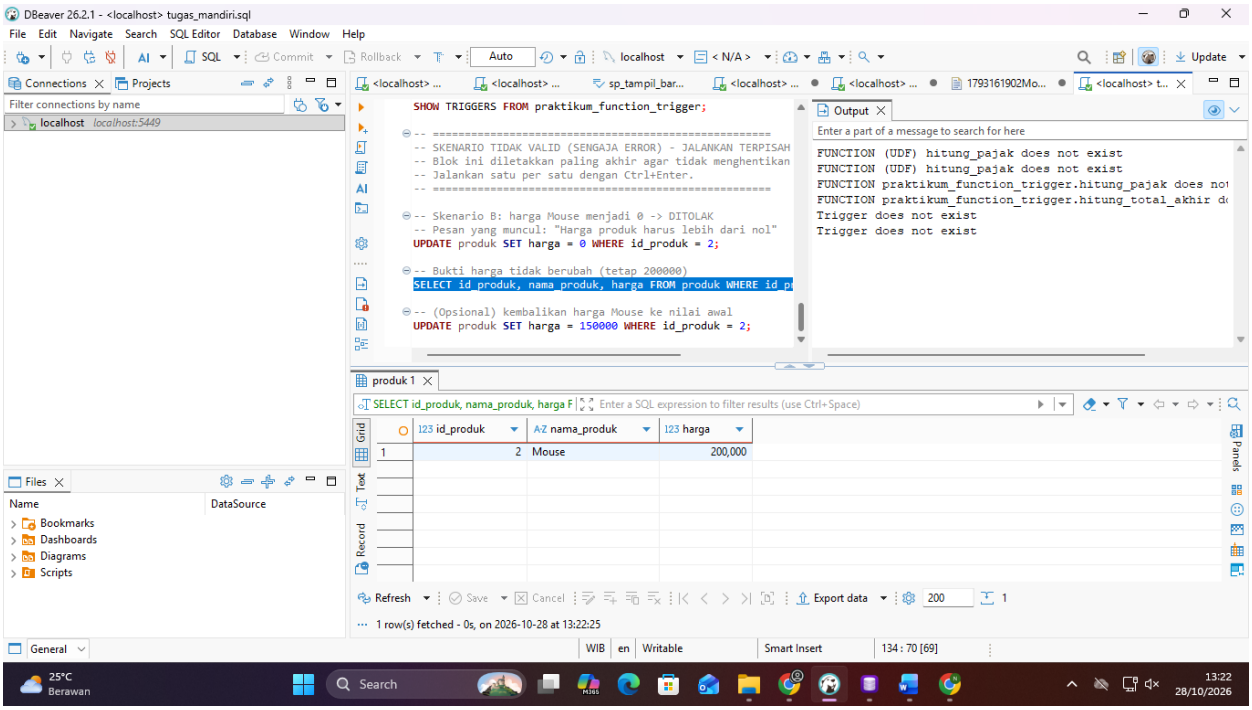
FOR EACH ROW

BEGIN

    IF NEW.harga <= 0 THEN

        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'Harga produk harus lebih dari nol';
    END IF;
END //
DELIMITER ;
```




```
DROP TRIGGER IF EXISTS validasi_harga_produk;
DELIMITER //
CREATE TRIGGER validasi_harga_produk
BEFORE UPDATE ON produk
FOR EACH ROW
BEGIN
    IF NEW.harga <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Harga produk harus lebih dari nol';
    END IF;
END //
```

DELIMITER ;

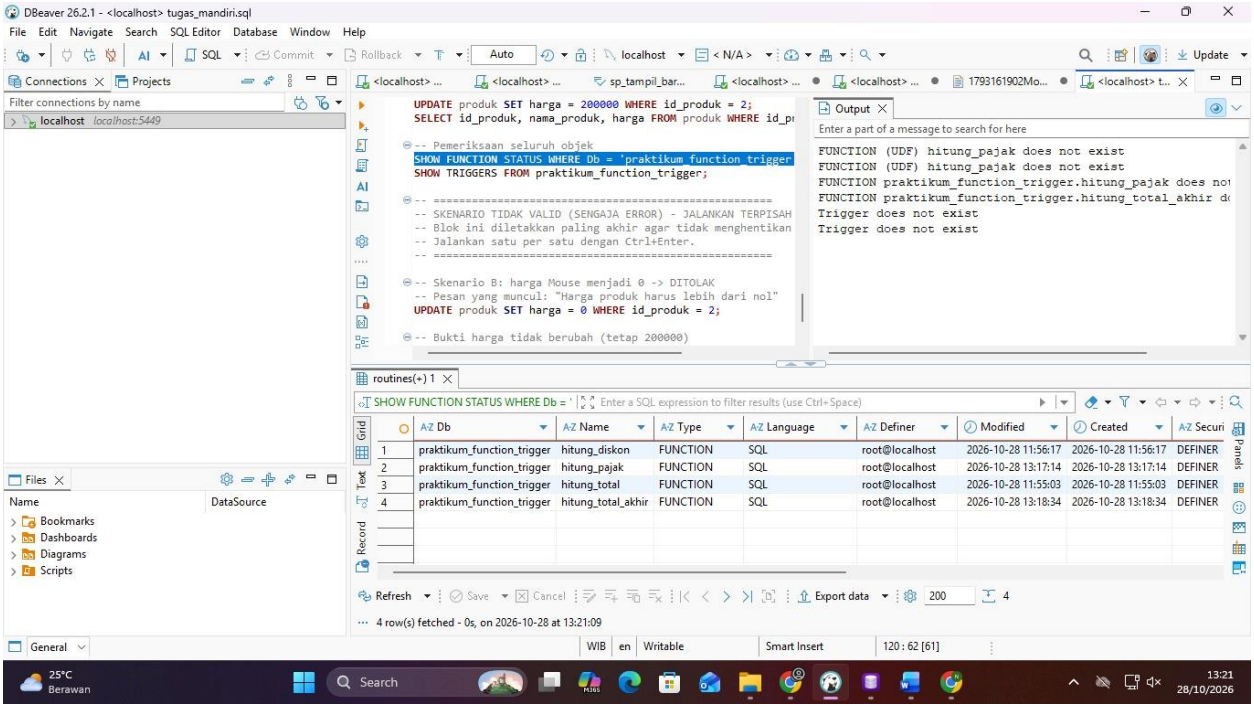
Skenario A (valid): harga Mouse menjadi 200000 -> DITERIMA

UPDATE produk SET harga = 200000 WHERE id_produk = 2;

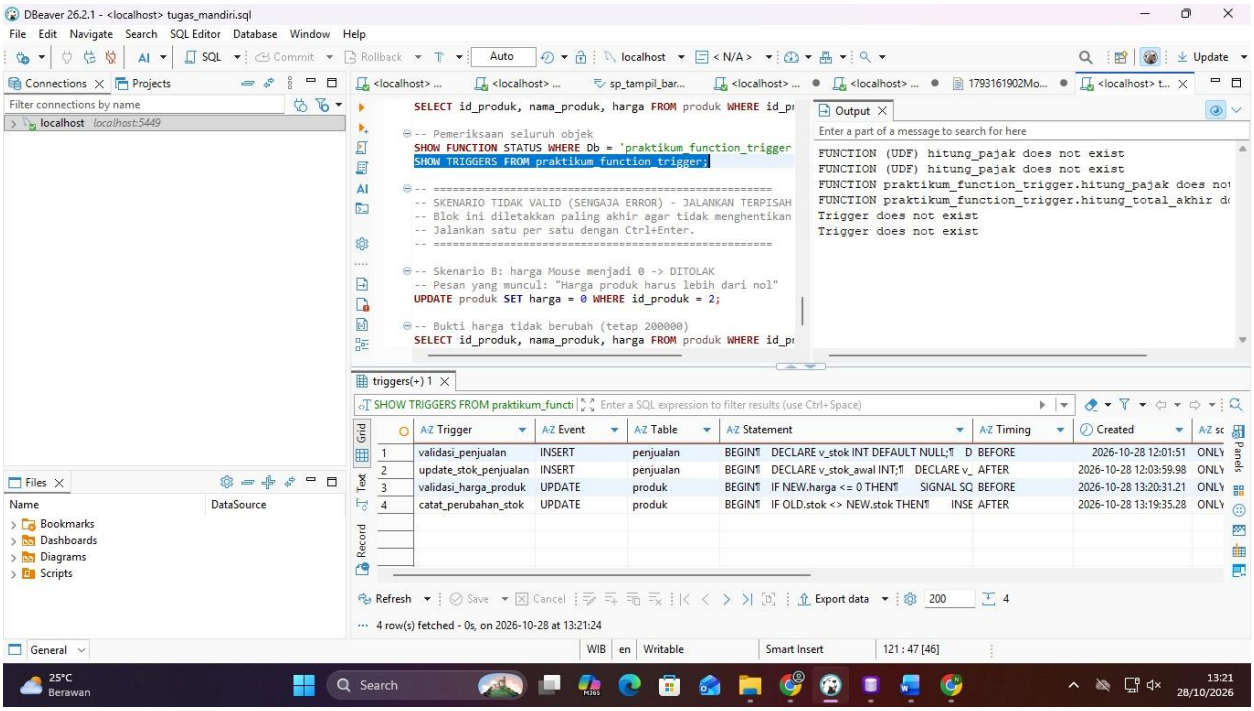
SELECT id_produk, nama_produk, harga FROM produk WHERE id_produk = 2;

Pemeriksaan seluruh objek

SHOW FUNCTION STATUS WHERE Db = 'praktikum_function_trigger';



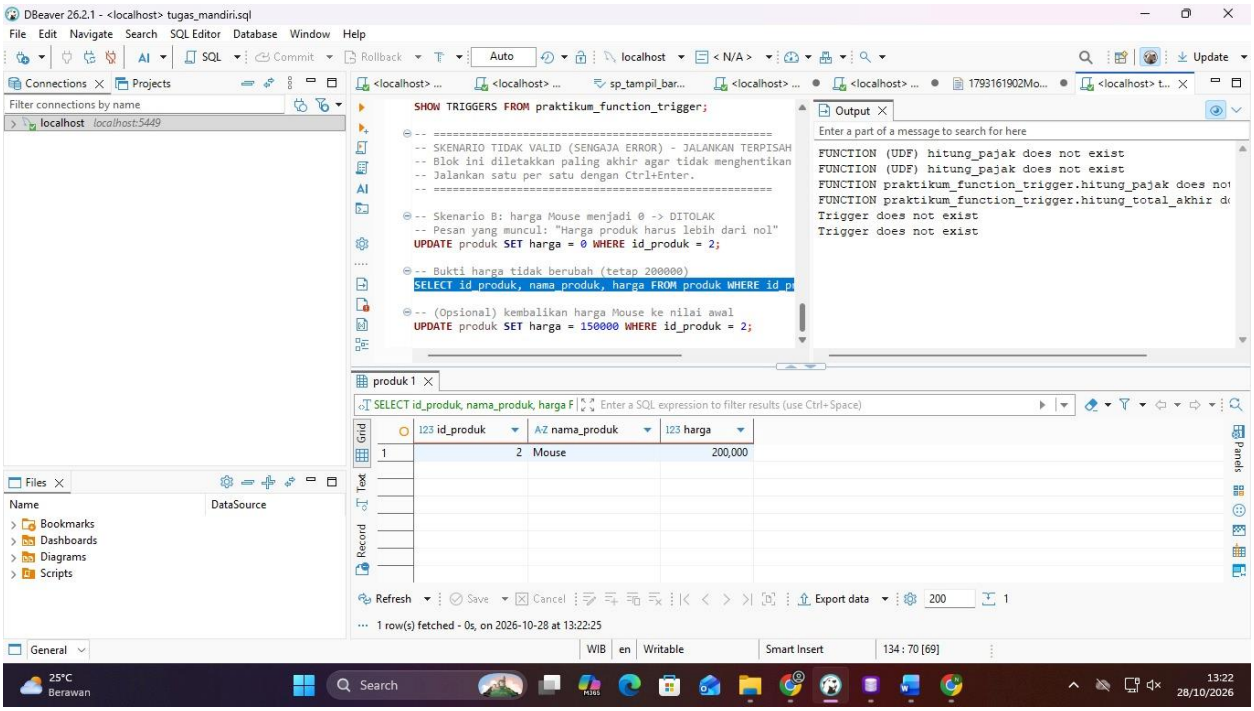
SHOW TRIGGERS FROM praktikum_function_trigger;



Uji dua skenario:

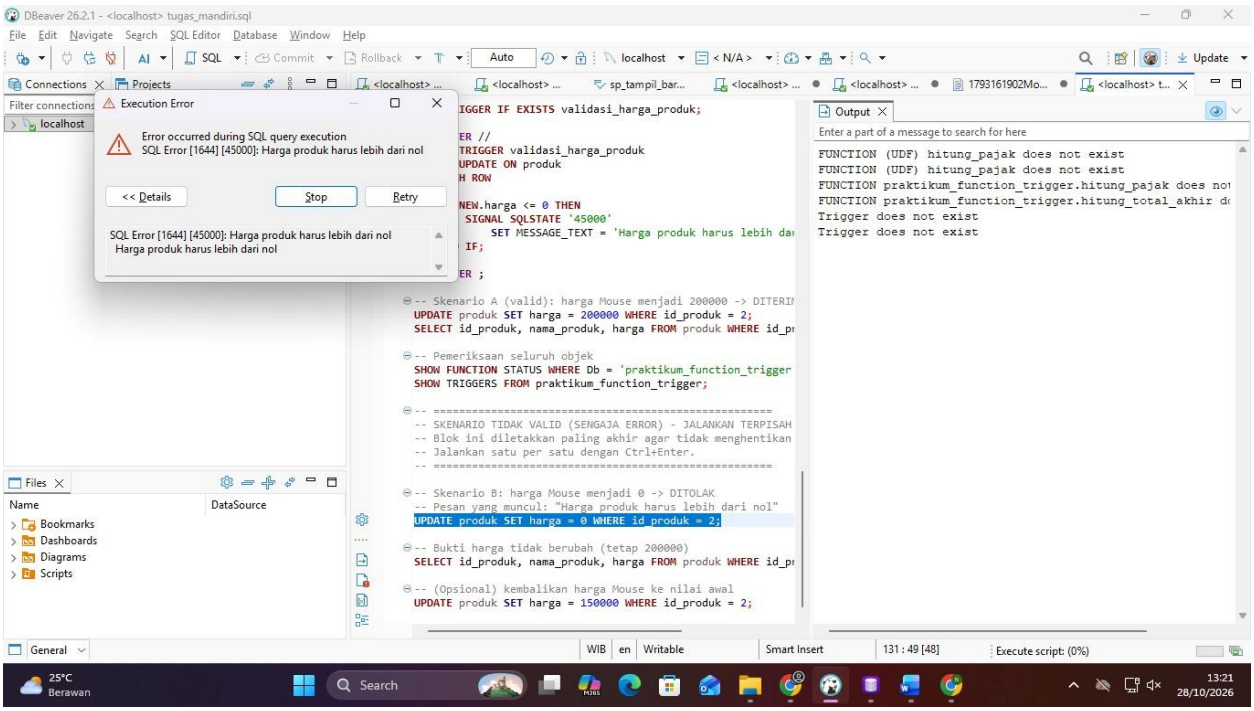
Mengubah harga Mouse menjadi Rp200.000: perubahan diterima.

SELECT id_produk, nama_produk, harga **FROM** produk **WHERE** id_produk = 2;



Mengubah harga Mouse menjadi Rp0: perubahan ditolak.

UPDATE produk **SET** harga = 0 **WHERE** id_produk = 2;



Tugas 5. Analisis Integrasi Function dan Trigger

Jawab pertanyaan berikut berdasarkan hasil implementasi:

1. Apa perbedaan function hitung_total dengan trigger validasi_penjualan?
2. Apa dampaknya jika trigger pembaruan stok tidak dibuat?
3. Mengapa perubahan stok perlu dicatat dalam tabel log?
4. Bagaimana function dan trigger membantu menjaga integritas data?
5. Apa yang perlu diperhatikan ketika dua transaksi mencoba membeli produk dengan stok terbatas secara bersamaan?

Jawaban :

1. Function hitung_total digunakan untuk menghitung total harga barang, sedangkan trigger validasi_penjualan digunakan untuk memeriksa transaksi sebelum disimpan.
   Contoh: Harga barang Rp250.000 dibeli 4 unit, totalnya Rp1.000.000. Jika jumlah pembelian nol, transaksi akan ditolak.

2. Stok tidak akan berkurang otomatis setelah penjualan sehingga data stok bisa tidak sesuai dengan kondisi sebenarnya.

   Contoh: Stok Mouse awalnya 25 unit. Setelah terjual 2 unit, stok seharusnya menjadi 23 unit.

3. Supaya riwayat perubahan stok bisa diketahui dan diperiksa kembali jika terjadi kesalahan.

   contoh: Pada tugas mandiri, stok Mouse diubah dari 25 menjadi 30 unit. Perubahan tersebut dicatat dalam tabel log_stok menggunakan OLD.stok sebagai stok awal dan NEW.stok sebagai stok akhir

4. Function membantu melakukan perhitungan secara otomatis, sedangkan trigger memvalidasi perubahan data agar sesuai dengan aturan yang ditentukan.

   Contoh: Function hitung_pajak menghitung pajak 11% dari transaksi Rp2.000.000, yaitu Rp220.000. Sementara itu, trigger validasi harga menolak perubahan harga produk menjadi Rp0.

5. Sistem harus memastikan stok mencukupi dan mengatur transaksi agar tidak terjadi penjualan melebihi persediaan.

   Contoh: Jika stok Laptop tersisa 3 unit dan dua pelanggan masing-masing membeli 2 unit secara bersamaan, sistem harus mengatur transaksi agar tidak terjadi kekurangan stok.